Plan del proyecto: App de gestión interna para pizzería

Oct 16, 2026 · @Rodrigo slucka

Resumen y alcance

Construiremos una app web instalable (PWA) hecha con React y pensada primero para el celular, que reemplaza papel, WhatsApp y planillas en la operación diaria de la pizzería: tomar pedidos, pasarlos a cocina, controlar stock y cerrar la caja.

Quién la usa

Rol	Qué hace desde el celular
Dueño / administrador	Ve el balance de caja, ventas, costos, stock y rendimiento del personal; configura menú y precios
Cajero / encargado	Cobra, abre y cierra caja, gestiona pedidos de delivery y retiro
Mozo	Toma pedidos en mesa y los envía a cocina
Cocina / pizzero	Ve las comandas en orden y marca cada pedido como en preparación o listo
Repartidor	Ve sus entregas asignadas y marca el estado

Qué problemas resuelve

- Pedidos que se pierden o se anotan mal entre salón, mostrador y cocina.
- Falta de control del stock de masa, queso y bebidas, que genera faltantes o mermas.
- Cierres de caja manuales y sin trazabilidad.
- Sin datos para decidir: qué se vende más, en qué horarios, cuánto se gana por producto.

Fuera del alcance inicial: impresión de tickets o comandas (todo es digital en celulares), pedidos online para clientes finales, pasarelas de pago y facturación electrónica. Se pueden sumar después sobre la misma base.

Módulos y funcionalidades por fases

El MVP cubre lo que la pizzería necesita para operar un día completo sin papel: pedidos, cocina y caja. Lo demás llega en fases posteriores.

Fase	Módulo	Funcionalidades clave
MVP	Acceso y roles	Login por usuario, permisos por rol (dueño, cajero, mozo, cocina, repartidor)
MVP	Menú y precios	Pizzas, tamaños, variantes, extras, bebidas; activar o pausar productos
MVP	Pedidos	Mesa, mostrador, retiro y delivery; notas por ítem; estados del pedido
MVP	Pantalla de cocina	Comandas en tiempo real, orden por llegada, aviso sonoro, marcar listo
MVP	Caja	Apertura y cierre, cobro por método de pago, balance de caja en vivo (ingresos, egresos y efectivo esperado), cuadre y resumen diario
MVP	Mesas	Plano de salón, cuentas por mesa, dividir y unir cuentas
MVP	Delivery	Asignación a repartidor, dirección y teléfono del cliente, seguimiento de estado
Fase 2	Inventario	Stock de insumos, recetas por pizza, descuento automático, alertas de mínimo
Fase 2	Clientes	Ficha con historial y dirección frecuente
Fase 3	Reportes	Ventas por día, hora y producto; margen; ticket promedio; exportar a Excel o PDF
Fase 3	Personal	Turnos, horas trabajadas, propinas
Fase 3	Integraciones	Pedidos online, WhatsApp y facturación si más adelante la necesitas

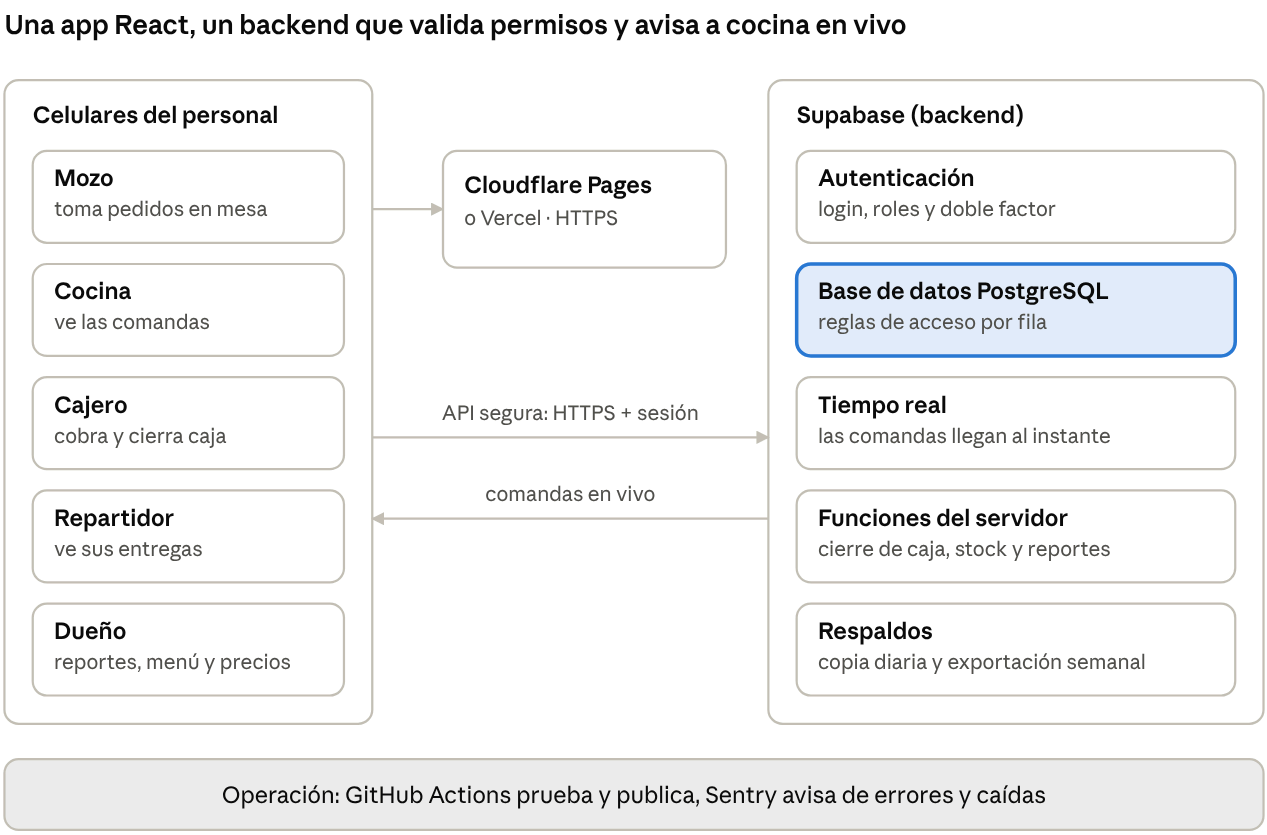
Stack tecnológico recomendado

Recomiendo React + TypeScript como PWA sobre una base de datos PostgreSQL administrada con tiempo real incluido. Es el camino más corto al MVP, con poco mantenimiento de servidores, y no te ata: la base es PostgreSQL estándar y se puede migrar.

Capa	Tecnología	Por qué
Frontend	React + TypeScript + Vite	Rápido, tipado y con ecosistema amplio
Instalable y offline	PWA (vite-plugin-pwa, service worker)	Se instala desde el navegador, sin tiendas de apps
Estilos y UI	Tailwind CSS + componentes accesibles (Radix o shadcn/ui)	Diseño mobile-first, botones grandes, rápido de iterar
Estado y datos	TanStack Query + Zustand	Caché, reintentos y estado local simple
Navegación	React Router	Rutas por rol
Formularios	React Hook Form + Zod	Validación en cliente y servidor
Backend y base de datos	Supabase (PostgreSQL, Auth, Realtime, Storage)	Autenticación, permisos por fila y comandas en vivo sin armar servidor propio
Lógica del servidor	Funciones edge de Supabase para cierres de caja, descuento de stock y reportes	Reglas críticas fuera del cliente
Pruebas	Vitest, Testing Library, Playwright	Unitarias, de componentes y de extremo a extremo en móvil
Calidad	ESLint, Prettier, GitHub Actions	Integración continua

Alternativa si prefieres control total: backend propio con Node.js (Fastify o NestJS) y PostgreSQL en un VPS. Da más control y costo fijo, pero suma trabajo de operación, seguridad y actualizaciones que recaerían en ti.

Arquitectura y diseño mobile-first



arquitectura · 5 roles, hosting, backend y operación

Todos los roles usan la misma app instalada en el celular; el hosting solo entrega el código, y cada dato y permiso pasa por Supabase.

Modelo de datos básico: usuarios y roles, productos y variantes, pedidos y sus ítems, mesas, pagos, cajas, insumos y recetas, clientes.

Diseño pensado para el celular

- Botones y zonas táctiles grandes (mínimo 44 px), uso con una mano y con manos mojadas o con harina.
- Pantallas por rol: cada persona ve solo lo que necesita, en un máximo de tres toques para tomar un pedido.
- Modo oscuro y alto contraste para cocina y turnos nocturnos.
- Carga rápida con la app instalada y caché de menú y precios.

Sin internet: el mozo puede seguir tomando pedidos y estos se envían al volver la conexión. Las comandas en cocina requieren conexión, por eso el plan de contingencia incluye papel de respaldo.

Avisos: alerta sonora y vibración en la pantalla de cocina cuando entra un pedido. Las notificaciones push al celular se agregan en la fase 2.

Hosting e infraestructura

Para una pizzería, la opción más equilibrada cuesta entre 25 y 50 USD al mes: frontend en una red de distribución global y base de datos administrada con copias de seguridad diarias. Los precios son aproximados y de memoria; confírmalos en cada proveedor antes de contratar.

Opción	Qué incluye	Costo mensual aprox. (USD)	Ventaja	Límite
A. Cloudflare Pages o Vercel + Supabase Pro (recomendada)	Frontend global, base de datos, auth, tiempo real, backups diarios	25 a 45	Casi sin mantenimiento, escala sola	Dependes de proveedores externos
B. VPS único (Hetzner, DigitalOcean) con Docker, Node y PostgreSQL	Todo en un servidor que administras tú	6 a 20	Costo bajo y fijo, control total	Tú actualizas, aseguras y respaldas
C. Nube gestionada (AWS, GCP, Azure)	Servicios separados y escalables	40 a 100 o más	Máxima flexibilidad	Complejidad y costo excesivos para este tamaño

Dominio y correo: un dominio propio cuesta unos 10 a 15 USD al año. Se registra en un proveedor con autenticación en dos pasos y renovación automática.

Entornos

- Desarrollo: en tu equipo, con base de datos local o proyecto separado.

- Staging: copia idéntica a producción con datos de prueba, para validar antes de publicar.
- Producción: solo recibe código que pasó por staging.

Despliegue continuo (CI/CD): cada cambio en GitHub ejecuta pruebas y análisis de calidad; si pasan, se publica solo a staging, y a producción con aprobación manual.

Respaldos: copia diaria automática de la base de datos con retención de al menos 7 días, más una exportación semanal guardada fuera del proveedor principal. Se prueba restaurarla antes de salir a producción.

Seguridad

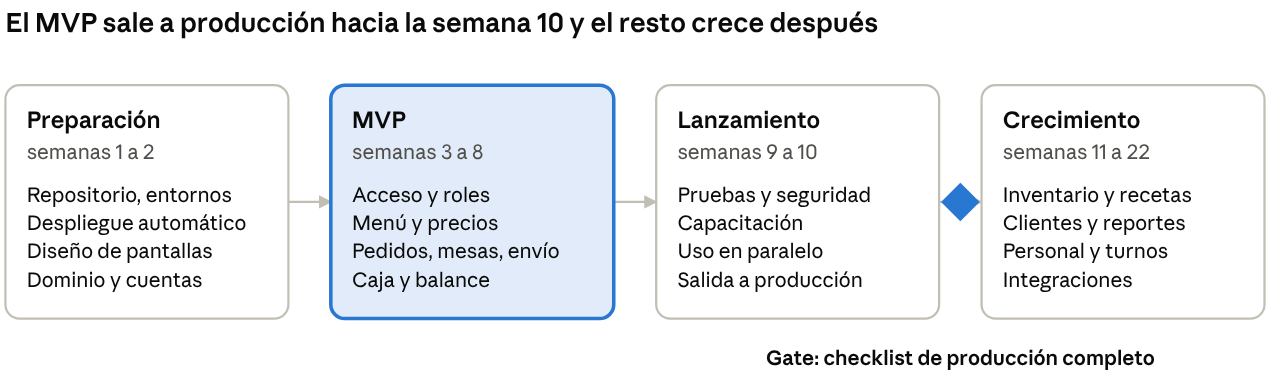
La regla central es que el celular nunca decide qué se puede hacer: cada permiso se valida en el servidor y en la base de datos. Con eso y el resto de controles de la tabla, la app queda protegida frente a los riesgos habituales de un sistema interno.

Área	Medida	Cuándo
Autenticación	Usuario y contraseña robustos, sesiones que expiran, doble factor obligatorio para el dueño y los administradores	MVP
Autorización	Roles con mínimo privilegio y reglas de seguridad por fila en PostgreSQL (un mozo no ve reportes, costos ni el balance de caja)	MVP
Transporte	HTTPS en todo, redirección forzada y encabezados de seguridad (HSTS, CSP, X-Frame-Options)	MVP
Datos	Cifrado en reposo del proveedor, solo los datos personales imprescindibles de los clientes	MVP
Secretos	Claves y tokens solo en variables de entorno del proveedor, nunca en el código ni en el repositorio; la clave de servicio jamás llega al navegador	MVP

Área	Medida	Cuándo
Validación	Validación de entradas con Zod en cliente y servidor; consultas parametrizadas contra inyección SQL	MVP
Dependencias	Alertas automáticas de vulnerabilidades (Dependabot) y actualización periódica	MVP
Auditoría	Registro de quién anuló un pedido, cambió un precio o cerró caja, con fecha y hora	Fase 2
Límites de abuso	Límite de intentos de login y de peticiones por usuario	MVP
Respaldo y recuperación	Copias diarias, restauración probada y un plan escrito de recuperación	Antes de producción
Dispositivos	Cierre de sesión automático por inactividad y baja inmediata de usuarios que dejan el negocio	MVP

Datos personales: si guardas teléfonos y direcciones de clientes, conviene una política de privacidad breve y cumplir la ley de protección de datos de tu país. Es un punto para revisar con un profesional legal.

Plan de desarrollo por etapas



plan por etapas · 4 fases, 1 gate

La app no sale a producción hasta cumplir el checklist de la sección siguiente. Los tiempos son una estimación para una persona desarrollando a tiempo completo y se ajustan cuando confirmemos alcance y equipo.

Checklist de puesta en producción

No se publica hasta marcar todo lo siguiente.

Calidad

- [x] ~~Pruebas automáticas de los flujos críticos: tomar pedido, enviar a cocina, cobrar, cerrar caja~~
- [ ] Pruebas manuales en al menos dos celulares reales, uno Android económico y un iPhone
- [ ] Prueba con conexión lenta y con corte de internet
- [ ] Prueba de carga con el pico esperado de pedidos simultáneos

Seguridad

- [x] ~~Doble factor activo en las cuentas de administrador, del proveedor y del dominio~~
- [ ] Reglas de acceso por fila probadas con cada rol
- [ ] Ningún secreto en el repositorio; claves de prueba rotadas
- [ ] Restauración de un respaldo probada de punta a punta

Infraestructura

- [x] ~~Dominio propio con HTTPS válido y renovación automática~~
- [ ] Staging y producción separados
- [ ] Despliegue automático con aprobación manual a producción
- [ ] Monitoreo de errores (Sentry) y de disponibilidad con alertas al celular del dueño

Operación

- [ ] Capacitación de cada rol con práctica real, de 30 a 60 minutos por rol
- [ ] Guía rápida impresa para el personal
- [ ] Periodo de uso en paralelo con el método anterior durante una semana
- [ ] Plan de contingencia: qué hacer si falla internet o la app (comandas en papel como respaldo)
- [ ] Responsable del soporte y canal para reportar problemas

Después del lanzamiento: revisión semanal de errores y comentarios durante el primer mes, y actualización mensual de dependencias.

Decisiones pendientes y próximos pasos

Con tus respuestas el alcance queda definido: un solo local con mesas, mostrador y delivery, todo en celulares y sin impresión. Quedan tres puntos por confirmar antes de programar.

1. **Tamaño del negocio:** 1 local.
2. **Modalidades de venta:** las 3 opciones delivery, mostrador y mesas para comer.
3. **Cobros y facturación:** solo llevar un historial de ventas para luego hacer el cierre de caja y cuadre.
4. **Equipos:** tanto cocina como barra trabajaran con dispositivos movil.

Por confirmar

- **Métodos de pago:** efectivo, QR, tarjeta u otros, para que el balance de caja los muestre por separado.
- **Quién ve el balance de caja:** asumí que solo el dueño y el cajero; dime si debe verlo otro rol.
- **Backend:** Supabase (lo asumido) o servidor propio, según el comentario en la sección de stack.

Próximos pasos una vez resueltas

1. Confirmar el stack y el hosting recomendados.
2. Crear el repositorio, los entornos y el pipeline de despliegue.
3. Diseñar las pantallas del MVP, empezando por toma de pedido y cocina.
4. Desarrollar y probar el MVP con el personal real.